DS9 Integrated Source Extractor

Technical Reference Manual

SEP-Based Source Detection, Photometry,
and Catalog Management
Integrated into SAOImageDS9

Juhan Kim

March 2026

Based on SAOImageDS9 (Smithsonian Astrophysical Observatory)
and SEP — Source Extraction and Photometry (Barbary et al.)

Contents

I User Guide

1 Quick Start Guide . . . 9
1.1 Overview . . . 9
1.2 Prerequisites . . . 9
1.3 Building DS9 with the Source Extractor . . . 10
1.4 Using the Source Extractor . . . 10
1.4.1 Step 1: Launch DS9 . . . 10
1.4.2 Step 2: Extract Sources . . . 10
1.4.3 Step 3: Browse the Catalog . . . 11
1.4.4 Step 4: Filter Results . . . 11
1.4.5 Step 5: Clear and Re-extract . . . 11
1.5 Interface Layout . . . 11
1.6 Troubleshooting . . . 12

2 Architecture . . . 13
2.1 Modified Source Files . . . 13
2.2 Top-Level Window Structure . . . 13
2.3 Data Flow . . . 14

3 Extraction Algorithm . . . 15
3.1 Background Estimation . . . 15
3.2 Source Detection . . . 15
3.2.1 Deblending . . . 16
3.3 Photometry . . . 16
3.3.1 AUTO Photometry (Kron Elliptical Aperture) . . . 16
3.3.2 Isophotal Corrected Photometry . . . 16
3.3.3 Aperture Photometry . . . 17
3.3.4 Magnitude Conversion . . . 17
3.4 Morphological Parameters . . . 17
3.4.1 Shape Parameters . . . 17
3.4.2 Half-Light Radius . . . 17
3.5 Surface Brightness . . . 18
3.6 Star/Galaxy Classification . . . 18
3.7 Flags . . . 18
3.8 WCS Coordinate Transformation . . . 19

4 Output Parameters 21
4.1 Complete Parameter Reference 21

5 Command-Line Reference 23
5.1 Synopsis 23
5.2 Options 23
5.3 Usage Examples 24

II Developer Reference

6 Tcl/Tk Implementation Details 29
6.1 Catalog Panel Procedures 29
6.1.1 CreateCatalogPanel 29
6.1.2 CatalogPanelExtract 29
6.1.3 CatalogPanelLoadTSV 30
6.1.4 CatalogPanelFilter 30
6.1.5 CatalogPanelAutoExtract 30
6.2 Global Variables 30

7 Python Extractor Details 31
7.1 Module Structure 31
7.2 FITS File Handling 31
7.3 Automatic Header Parsing 31
7.4 Dependencies 32

8 Image–Catalog Interaction 33
8.1 Overview of Interaction Features 33
8.1.1 Title Bar Layout 33
8.2 Mark All Sources 34
8.3 Marker Click — Image to Catalog Navigation 34
8.3.1 Implementation 34
8.4 Visible Filter 35
8.4.1 Viewport Calculation 35
8.5 Source Merging (Ctrl+Click + Ctrl+M) 35
8.5.1 Step 1: Select Merge Candidates 35
8.5.2 Step 2: Execute Merge (Ctrl+M) 35
8.5.3 Cancel (Escape) 36
8.6 Trim — Column Filter Dialog 36
8.6.1 Dialog Layout 36
8.7 Settings Dialog 37
8.8 Marker Tag Reference 37
8.9 Key Bindings 37

9 Building `ds9_sextract` for Each Platform 39
9.1 Dependencies 39
9.2 Linux (x86_64) 39
9.3 macOS (Intel and Apple Silicon) 40
9.4 Windows 40
9.4.1 Using MSYS2/MinGW-w64 40
9.4.2 Using Visual Studio 41
9.5 Cross-Platform Tcl Integration 41
9.6 Verification 42

User Guide

1 Quick Start Guide

This chapter provides a step-by-step guide to using the integrated Source Extractor in DS9. For detailed explanations of the extraction algorithm, output parameters, and internal architecture, refer to the subsequent chapters.

1.1 Overview

The DS9 Integrated Source Extractor adds SExtractor-like source detection and photometry capabilities directly within the SAOImageDS9 image viewer. When a FITS image is loaded, sources can be extracted with a single click, and the resulting catalog is displayed in a resizable panel alongside the image.

Key features:

- **One-click extraction** — Press the "Extract" button to detect and measure all sources in the current FITS image
- **Split-panel interface** — The main window is divided into an image viewer (left) and a catalog table (right), separated by a draggable divider
- **19 output parameters** — Photometry, astrometry, morphology, surface brightness, and star/galaxy classification
- **Real-time filtering** — Filter the catalog interactively by any column value
- **Automatic WCS** — Celestial coordinates (RA, Dec) are computed automatically when WCS information is available in the FITS header
- **Auto-extraction** — Sources are extracted automatically when a new FITS file is loaded

1.2 Prerequisites

- **DS9** — SAOImageDS9 built with the integrated Source Extractor patch
- **Python 3** with the following packages:
  - `sep` — Source Extraction and Photometry library (Python binding of SExtractor)
  - `astropy` — FITS I/O and WCS coordinate transformations
  - `numpy` — Numerical array operations
- `ds9_sextract` **binary** — The compiled Python script, located in the same directory as the `ds9` executable

Install Python Dependencies

```
pip install sep astropy numpy
```

1.3 Building DS9 with the Source Extractor

Build Commands

```
cd SAOImageDS9
make -C ds9/unix -j 128
```

The build process compiles the C/Tcl core, packages the Tcl library files (including the modified `layout.tcl` and `ds9.tcl`) into `ds9.zip`, and produces the final `bin/ds9` executable. The `ds9_sextract` binary must be placed in the same `bin/` directory.

Important

After modifying any `.tcl` file in `ds9/library/`, you must copy it to `ds9/unix/mntpt/library/`, update the zip, and rebuild:

```
cp ds9/library/layout.tcl ds9/unix/mntpt/library/
cp ds9/library/ds9.tcl    ds9/unix/mntpt/library/
cd ds9/unix
zip -ru ds9.zip mntpt/library/layout.tcl mntpt/library/ds9.
    tcl
cat ds9base ds9.zip > ds9
zip -A ds9
chmod 755 ds9
cp ds9 ../../bin/
```

1.4 Using the Source Extractor

1.4.1 Step 1: Launch DS9

Launch

```
./bin/ds9 image.fits
```

The window opens with two panels separated by a vertical divider:

- **Left panel** — The standard DS9 interface (menu bar, info panel, buttons, image canvas, colorbar)
- **Right panel** — The Source Extractor catalog panel (title bar, filter bar, data table, status bar)

The divider can be dragged left or right to resize the panels.

1.4.2 Step 2: Extract Sources

Click the **"Extract"** button in the catalog panel title bar. The status bar will show progress:

1. `Extracting sources from image.fits ...` — Extraction is running
2. `image.fits: 142 sources extracted` — Extraction complete

The catalog table is automatically populated with all detected sources, sorted by brightness (brightest first).

Auto-Extraction

When a FITS file is loaded via `File` → `Open`, sources are extracted automatically after a 500 ms delay. This behavior is controlled by the `CatalogPanelAutoExtract` hook in `fits.tcl`.

1.4.3 Step 3: Browse the Catalog

The catalog table displays 19 columns (see Chapter 4 for detailed descriptions):

Category	Columns
Identification	NUMBER
Astrometry	X_IMAGE, Y_IMAGE, ALPHA_J2000, DELTA_J2000
Photometry	MAG_AUTO, MAG_ISOCOR, MAG_APER, FLUX_AUTO
Morphology	FLUX_RADIUS, A_IMAGE, B_IMAGE, THETA_IMAGE, ELLIPTICITY KRON_RADIUS
Surface Brightness	MU_MAX, MU_THRESHOLD
Classification	CLASS_STAR, FLAGS

- Scroll horizontally and vertically using the scrollbars
- Column widths can be resized by dragging column borders in the header row
- Rows can be selected by clicking (extended selection with Shift/Ctrl)

1.4.4 Step 4: Filter Results

Type a search pattern in the **Filter** entry and click **"Apply"** (or press Enter). The filter performs case-insensitive substring matching across all columns. For example:

- Type `0.9` to show only rows containing "0.9" in any field
- Clear the filter field and press Apply to show all sources again

1.4.5 Step 5: Clear and Re-extract

Click **"Clear"** to reset the table and status. Load a new FITS image and click **"Extract"** again, or rely on auto-extraction.

1.5 Interface Layout

The window hierarchy below the pulldown menu bar:

Widget Hierarchy

```
. (top-level window)
|-- .mb (pulldown menu bar: File, Edit, View, ...)
|-- .pw (ttk::panedwindow, horizontal, draggable divider)
    |-- .ds9 (left pane: main DS9 interface)
    |    |-- header (info panel, magnifier, panner)
    |    |-- buttons (toolbar)
    |    |-- image frame
    |        |-- canvas (FITS image display)
    |        |-- colorbar, graphs
    |-- .catf (right pane: catalog panel)
         |-- titlebar ("Source Extractor", Extract, Clear)
```

```
            |-- searchbar (Filter entry + Apply)
            |-- table (19-column scrollable data table)
            |-- statusbar (status messages)
```

The `ttk::panedwindow` widget provides a sash (divider handle) that the user can drag to redistribute space between the image viewer and the catalog panel. The left pane has weight 3 and the right pane has weight 2, giving an initial 60:40 split.

1.6 Troubleshooting

Symptom	Solution
`ds9_sextract not found`	Ensure the `ds9_sextract` binary is in the same directory as the `ds9` executable. Check that it has execute permission (`chmod +x`).
`No FITS image loaded`	Open a FITS file before clicking Extract. The extractor requires a frame with a loaded FITS image.
`Extraction error: ...`	Check that Python dependencies (`sep`, `astropy`, `numpy`) are installed. Check `LD_LIBRARY_PATH` if using conda.
`No sources detected`	The image may have very low signal-to-noise. Try adjusting the detection threshold (default: 1.5σ) via command-line arguments to `ds9_sextract`.
Catalog panel not visible	Drag the divider from the right edge of the window toward the center. The panel may have been collapsed.
WCS coordinates show 0.0	The FITS header does not contain valid WCS keywords. Pixel coordinates (X_IMAGE, Y_IMAGE) are still accurate.

2 Architecture

This chapter describes the internal architecture of the integrated Source Extractor, covering the modified DS9 source files, the extraction pipeline, and the data flow between components.

2.1 Modified Source Files

Three files implement the integration:

File	Role
`ds9/library/ds9.tcl`	Creates the top-level `ttk::panedwindow` that splits the main window into two panes. Initializes `ds9(toppw)`, `ds9(main)`, and `ds9(catalog_frame)`. Calls `CreateCatalogPanel`.
`ds9/library/layout.tcl`	Defines `CreateCanvas` (image canvas, restored to original structure), `CreateCatalogPanel` (catalog UI), `CatalogPanelExtract` (runs `ds9_sextract`), `CatalogPanelLoadTSV` (parses output into table), `CatalogPanelFilter` (interactive filtering), and `CatalogPanelAutoExtract` (auto-extraction hook).
`ds9/library/fits.tcl`	Calls `CatalogPanelAutoExtract` after a FITS file is loaded, triggering automatic source extraction.

2.2 Top-Level Window Structure

The key architectural change is the introduction of a `ttk::panedwindow` at the highest level of the widget hierarchy, directly below the native pulldown menu bar.

ds9.tcl — Top-level panedwindow creation

```
# create top-level panedwindow
set ds9(toppw) [ttk::panedwindow ${ds9(top)}pw -orient
    horizontal]
pack $ds9(toppw) -fill both -expand true

# left pane: main DS9 content
set ds9(main) [ttk::frame ${ds9(top)}ds9]
```

```
$ds9(toppw) add $ds9(main) -weight 3

# right pane: catalog panel
set ds9(catalog_frame) [ttk::frame ${ds9(top)}catf]
$ds9(toppw) add $ds9(catalog_frame) -weight 2
```

This design ensures that:

1. The pulldown menu bar spans the full window width (it is set via `$ds9(top) configure -menu $ds9(mb)`, independent of pack geometry)
2. Everything below the menu bar is split into two resizable panes
3. All existing DS9 layout modes (horizontal, vertical, basic, advanced) continue to work within the left pane without modification
4. The catalog panel occupies its own independent pane on the right

2.3 Data Flow

The extraction pipeline follows this sequence:

1. **Trigger** — User clicks "Extract" (or auto-extraction fires after FITS load)
2. **File discovery** — `CatalogPanelExtract` retrieves the current FITS filename from the active frame via `$current(frame) get fits file name full`
3. **External process** — The `ds9_sextract` binary is executed as a child process via `exec sh -c`. The `LD_LIBRARY_PATH` is configured to include conda library paths.
4. **Output capture** — `ds9_sextract` writes a tab-separated table to `stdout`. The Tcl `exec` command captures this output as a string.
5. **Table population** — `CatalogPanelLoadTSV` parses the TSV string, splits it into rows and columns, and populates the `Tktable` widget via the `catpaneltbldb` array variable.
6. **Status update** — The status bar displays the filename and number of detected sources.

> The extraction runs as a separate process, not within the DS9 Tcl interpreter. This prevents numerical library conflicts and allows the Python-based extractor to use its own dependencies (NumPy, SEP, Astropy) independently.

3 Extraction Algorithm

The `ds9_sextract` tool implements a SExtractor-compatible extraction pipeline using the SEP (Source Extraction and Photometry) library. This chapter describes each step of the algorithm in detail.

3.1 Background Estimation

The first step is to estimate and subtract the sky background. SEP uses the same algorithm as SExtractor:

1. The image is divided into a grid of rectangular meshes (default size: 64×64 pixels, controlled by `-back-size`)
2. In each mesh cell, the local background level is estimated using iterative $\kappa\sigma$-clipping
3. The resulting coarse background map is smoothed with a median filter (default kernel: 3×3 cells, controlled by `-back-filtersize`)
4. The smoothed map is interpolated to full image resolution using bicubic spline interpolation

Background estimation

```
bkg = sep.Background(data, bw=back_size, bh=back_size,
                     fw=back_filtersize, fh=back_filtersize)
data_sub = data - bkg.back()
```

Two outputs are produced:

- `bkg.back()` — the background level map $B(x,y)$
- `bkg.rms()` — the background RMS (noise) map $\sigma_B(x,y)$, used as the detection threshold and photometric error

3.2 Source Detection

Sources are detected as groups of connected pixels above a threshold:

$$I(x,y) - B(x,y) > k \cdot \sigma_B(x,y) \tag{3.1}$$

where k is the detection threshold in units of the local background RMS (default: $k = 1.5$, set by `-detect-thresh`). Connected pixel groups with fewer than `-detect-minarea` pixels (default: 5) are rejected.

3.2.1 Deblending

Overlapping sources are separated using multi-threshold deblending:

1. The detected object is re-thresholded at `-deblend-nthresh` (default: 32) exponentially spaced sub-thresholds between the detection threshold and the peak value
2. At each sub-threshold, connected components are identified
3. A component is split into a separate object if it contains more than `-deblend-mincont` (default: 0.005) of the total flux of the parent object

Source detection with deblending

```
objects, segmap = sep.extract(
    data_sub, detect_thresh, err=bkg_rms,
    minarea=detect_minarea,
    deblend_nthresh=deblend_nthresh,
    deblend_cont=deblend_mincont,
    segmentation_map=True
)
```

The segmentation map assigns each pixel to the object it belongs to (or zero for background), enabling morphological measurements.

3.3 Photometry

Three photometric methods are applied to each detected source.

3.3.1 AUTO Photometry (Kron Elliptical Aperture)

The AUTO magnitude follows the Kron (1980) method:

1. The first-moment radius (Kron radius r_K) is computed within an ellipse of semi-major axis $6a$:

$$r_K = \frac{\sum_i r_i I_i}{\sum_i I_i} \tag{3.2}$$

   where r_i is the elliptical distance of pixel i from the object center, and I_i is the background-subtracted intensity.
2. A minimum Kron radius of 3.5 pixels is enforced to prevent underestimation for compact sources.
3. The flux is summed within an elliptical aperture of semi-major axis $2.5\,r_K$, following the SExtractor convention.

Kron Factor

The factor of 2.5 applied to the Kron radius ensures that $\gtrsim$96% of the total flux is captured for a typical galaxy profile (Sérsic $n \approx 1$–4), while $\gtrsim$99% is captured for a Gaussian PSF.

3.3.2 Isophotal Corrected Photometry

The isophotal flux is the sum of all pixels belonging to the detection footprint. A correction factor accounts for the flux missed below the detection isophote, using the SExtractor formula:

$$F_{\text{isocor}} = \frac{F_{\text{iso}}}{1 - 0.196\,\alpha - 0.751\,\alpha^2}, \qquad \alpha = \frac{N_{\text{pix}} \cdot t_{\text{local}}}{F_{\text{iso}}} \tag{3.3}$$

where N_{pix} is the number of detection pixels, t_{local} is the local threshold, and F_{iso} is the raw isophotal flux.

3.3.3 Aperture Photometry

A fixed circular aperture of diameter `-phot-aperture` (default: 5.0 pixels) is centered on each source. The flux is summed within this circle, with partial pixel weighting at the boundary.

3.3.4 Magnitude Conversion

All fluxes are converted to magnitudes using:

$$m = -2.5\log_{10}(F) + m_0 \tag{3.4}$$

where m_0 is the magnitude zero-point (default: 25.0, set by `-mag-zeropoint`). Sources with non-positive flux are assigned $m = 99.0$.

3.4 Morphological Parameters

3.4.1 Shape Parameters

The semi-major axis (a), semi-minor axis (b), and position angle (θ) are derived from the second-order moments of the intensity distribution within the detection isophote:

$$\overline{x^2} = \frac{\sum_i I_i x_i^2}{\sum_i I_i} - \bar{x}^2 \tag{3.5}$$

$$\overline{y^2} = \frac{\sum_i I_i y_i^2}{\sum_i I_i} - \bar{y}^2 \tag{3.6}$$

$$\overline{xy} = \frac{\sum_i I_i x_i y_i}{\sum_i I_i} - \bar{x}\bar{y} \tag{3.7}$$

The ellipse parameters are:

$$a^2 = \frac{\overline{x^2} + \overline{y^2}}{2} + \sqrt{\left(\frac{\overline{x^2} - \overline{y^2}}{2}\right)^2 + \overline{xy}^2} \tag{3.8}$$

$$b^2 = \frac{\overline{x^2} + \overline{y^2}}{2} - \sqrt{\left(\frac{\overline{x^2} - \overline{y^2}}{2}\right)^2 + \overline{xy}^2} \tag{3.9}$$

$$\theta = \frac{1}{2}\arctan\left(\frac{2\overline{xy}}{\overline{x^2} - \overline{y^2}}\right) \tag{3.10}$$

The ellipticity is defined as:

$$e = 1 - \frac{b}{a} \tag{3.11}$$

3.4.2 Half-Light Radius

The half-light radius (`FLUX_RADIUS`) is the radius of a circular aperture centered on the source that encloses 50% of the AUTO flux. It is computed by integrating the flux profile outward from the center until the cumulative flux reaches $0.5 \times F_{\text{AUTO}}$.

3.5 Surface Brightness

Two surface brightness measurements are provided:

- **MU_MAX** — Surface brightness of the brightest pixel:

$$\mu_{\max} = -2.5\log_{10}\left(\frac{I_{\text{peak}}}{\Omega_{\text{pix}}}\right) + m_0 \tag{3.12}$$

  where I_{peak} is the peak pixel value above background and Ω_{pix} is the pixel area in arcsec^2.
- **MU_THRESHOLD** — Surface brightness at the detection threshold:

$$\mu_{\text{thresh}} = -2.5\log_{10}\left(\frac{t_{\text{local}}}{\Omega_{\text{pix}}}\right) + m_0 \tag{3.13}$$

3.6 Star/Galaxy Classification

An approximate star/galaxy classifier is implemented using the ratio of the measured FWHM to the seeing FWHM:

$$\text{FWHM}_{\text{image}} = 2\sqrt{\ln 2 \cdot (a^2 + b^2)} \tag{3.14}$$

$$\text{CLASS_STAR} = \frac{1}{1 + \exp\big(3 \cdot (r_{\text{FWHM}} - 1.5)\big)}, \qquad r_{\text{FWHM}} = \frac{\text{FWHM}_{\text{image}}}{\text{FWHM}_{\text{seeing}}/s} \tag{3.15}$$

where s is the pixel scale in arcsec/pixel and $\text{FWHM}_{\text{seeing}}$ is set by `-seeing-fwhm` (default: 3.0 arcsec).

Objects with CLASS_STAR ≈ 1 are point-like (stars), while CLASS_STAR ≈ 0 indicates extended sources (galaxies).

> This is a simple heuristic classifier, not the neural-network-based classifier used by SExtractor. For accurate star/galaxy separation, use dedicated tools such as `PSFEx` + SExtractor or machine learning classifiers.

3.7 Flags

The `FLAGS` column is a bitmask combining detection and photometry flags:

Bit	Value	Meaning
0	1	Object has neighbors (bright and close enough to bias photometry)
1	2	Object was originally blended with another
2	4	At least one pixel is saturated (peak + background $\geq$ `SATURATE`)
3	8	Object is truncated (too close to image boundary)
4	16	Aperture data incomplete or corrupted
5	32	Isophotal data incomplete or corrupted

Multiple flags can be set simultaneously. For example, `FLAGS = 3` means the object has neighbors (1) and was deblended (2).

3.8 WCS Coordinate Transformation

If the FITS header contains valid World Coordinate System (WCS) keywords (`CRPIX`, `CRVAL`, `CD` or `CDELT+CROTA`), pixel coordinates are converted to celestial coordinates (J2000 equatorial):

WCS transformation

```
wcs = WCS(header)
coords = wcs.pixel_to_world(objects['x'], objects['y'])
alpha_j2000 = coords.ra.deg     # Right Ascension in degrees
delta_j2000 = coords.dec.deg    # Declination in degrees
```

If WCS is unavailable, `ALPHA_J2000` and `DELTA_J2000` are set to 0.0.

4 Output Parameters

This chapter provides a detailed description of each column in the output catalog.

4.1 Complete Parameter Reference

Parameter	Unit	Description
`NUMBER`	—	Running object number, assigned after sorting by brightness (brightest = 1).
`X_IMAGE`	pixel	Object centroid x-coordinate in FITS convention (1-indexed). Computed as the intensity-weighted first moment.
`Y_IMAGE`	pixel	Object centroid y-coordinate in FITS convention (1-indexed).
`ALPHA_J2000`	deg	Right Ascension (J2000 equatorial) computed from pixel coordinates via WCS. Zero if no WCS is available.
`DELTA_J2000`	deg	Declination (J2000 equatorial) computed from pixel coordinates via WCS. Zero if no WCS is available.
`MAG_AUTO`	mag	Kron-like elliptical aperture magnitude. Uses an elliptical aperture with semi-major axis $= 2.5 \times r_{\mathrm{Kron}}$. Most reliable total magnitude for galaxies.
`MAG_ISOCOR`	mag	Isophotal magnitude corrected for flux lost below the detection threshold, using the SExtractor analytical correction formula.
`MAG_APER`	mag	Fixed circular aperture magnitude. Aperture diameter set by `-phot-aperture` (default: 5.0 pixels). Useful for comparing point sources.
`FLUX_AUTO`	counts	Total flux within the Kron elliptical aperture (in ADU or counts).

Parameter	Unit	Description
FLUX_RADIUS	pixel	Half-light radius: radius of a circular aperture enclosing 50% of FLUX_AUTO. Indicator of object size; correlates with effective radius R_e for galaxies.
A_IMAGE	pixel	Semi-major axis of the object ellipse, derived from second-order intensity moments.
B_IMAGE	pixel	Semi-minor axis of the object ellipse.
THETA_IMAGE	deg	Position angle of the major axis, measured counter-clockwise from the x-axis (FITS convention). Range: -90 to $+90$.
ELLIPTICITY	—	Ellipticity: $e = 1 - b/a$. Values range from 0 (circular) to 1 (infinitely elongated).
KRON_RADIUS	pixel	First-moment radius (Kron radius), used to scale the AUTO aperture. Minimum enforced value: 3.5 pixels.
MU_MAX	mag/arcsec^2	Peak surface brightness: magnitude of the brightest pixel divided by pixel area. Lower values = brighter peaks.
MU_THRESHOLD	mag/arcsec^2	Detection threshold surface brightness at the object position. Objects with $\mu_{\rm max} < \mu_{\rm thresh}$ are detected.
CLASS_STAR	—	Star/galaxy classifier output. Range 0.0 (extended/galaxy) to 1.0 (point-like/star). Based on FWHM ratio to seeing.
FLAGS	—	Extraction flags (bitmask). See Section 3 for flag definitions. Zero means clean photometry.

5 Command-Line Reference

The `ds9_sextract` tool can also be used as a standalone command-line program, independent of the DS9 GUI.

5.1 Synopsis

```
ds9_sextract [OPTIONS] <FITS_FILE>
```

The output is a tab-separated table printed to `stdout`. The first line is the header row containing column names; subsequent lines contain data rows sorted by `MAG_AUTO` (brightest first).

5.2 Options

Option	Default	Description
`-detect-thresh`	1.5	Detection threshold in units of the local background RMS (σ). Lower values detect fainter sources but increase false positives.
`-detect-minarea`	5	Minimum number of connected pixels above the threshold required to register a detection. Rejects cosmic ray hits and noise spikes.
`-deblend-nthresh`	32	Number of exponentially spaced sub-thresholds used for deblending overlapping sources. Higher values provide finer deblending.
`-deblend-mincont`	0.005	Minimum contrast ratio for deblending. A sub-component must contain at least this fraction of the parent flux to be split off. Lower values split more aggressively.
`-phot-aperture`	5.0	Diameter of the fixed circular aperture (in pixels) used for `MAG_APER` measurements.

Option	Default	Description
`-mag-zeropoint`	25.0	Photometric zero-point m_0 for magnitude conversion: $m = -2.5\log_{10}(F) + m_0$. Set this to the calibrated zero-point of your image.
`-gain`	0	Detector gain in e^-/ADU. Used for Poisson noise estimation in photometric errors. If 0, the tool attempts to read `GAIN`, `EGAIN`, or `CCDGAIN` from the FITS header.
`-pixel-scale`	1.0	Pixel scale in arcsec/pixel. Used for surface brightness calculations. If 1.0 and WCS is available, the scale is automatically derived from WCS.
`-seeing-fwhm`	3.0	Seeing FWHM in arcsec. Used for star/galaxy classification (`CLASS_STAR`).
`-back-size`	64	Background mesh size in pixels. Each mesh cell has dimensions `back-size` × `back-size`. Smaller values track finer background variations but may subtract source flux.
`-back-filtersize`	3	Median filter kernel size (in mesh cells) applied to the background map before interpolation. Smooths out mesh-to-mesh noise.

5.3 Usage Examples

Basic extraction

```
ds9_sextract image.fits > catalog.tsv
```

Deep extraction with calibrated zero-point

```
ds9_sextract --detect-thresh 1.0 --detect-minarea 3 \
             --mag-zeropoint 28.09 --gain 2.5 \
             --seeing-fwhm 0.8 --pixel-scale 0.03 \
             hst_acs_f814w.fits > deep_catalog.tsv
```

Conservative extraction (few false positives)

```
ds9_sextract --detect-thresh 3.0 --detect-minarea 10 \
             --deblend-mincont 0.01 \
             crowded_field.fits > clean_catalog.tsv
```

Fine background estimation for extended objects

```
ds9_sextract --back-size 128 --back-filtersize 5 \
             galaxy_cluster.fits > cluster_catalog.tsv
```

II

Developer Reference

6 Tcl/Tk Implementation Details

This chapter documents the Tcl procedures that implement the catalog panel and its integration with the DS9 GUI.

6.1 Catalog Panel Procedures

6.1.1 CreateCatalogPanel

Creates all widgets for the catalog panel within `ds9(catalog_frame)`:

Widget	Description
`titlebar`	Horizontal frame containing the "Source Extractor" label, "Extract" button, and "Clear" button.
`searchbar`	Filter entry with "Apply" button. The entry is bound to `<Return>` for keyboard activation.
`tblf` (table frame)	Contains the `Tktable` widget (`catpanel(tbl)`) with horizontal and vertical scrollbars. The table is configured with 19 columns, row selection mode, and resizable column borders.
`statusbar`	Displays status messages via the `catpanel(status)` variable.

6.1.2 CatalogPanelExtract

Orchestrates the extraction process:

1. Locates the `ds9_sextract` binary in the same directory as the DS9 executable
2. Retrieves the current FITS filename from the active frame
3. Constructs `LD_LIBRARY_PATH` to include conda libraries (both `$CONDA_PREFIX/lib` and `$HOME/miniconda3/lib`)
4. Executes `ds9_sextract` via `exec sh -c` and captures stdout
5. Passes the TSV output to `CatalogPanelLoadTSV`

6.1.3 CatalogPanelLoadTSV

Parses tab-separated data into the Tktable widget:

1. Splits the input string by newlines into rows
2. Parses the first row as column headers
3. Configures the table dimensions (`-cols`, `-rows`)
4. Fills the `catpaneltbldb` array variable: `catpaneltbldb(`*row*, *col*`)`
5. Stores the raw data in `catpanel(alldata)` for filtering

6.1.4 CatalogPanelFilter

Performs case-insensitive substring filtering:

1. Reads the filter pattern from `catpanel(search_var)`
2. Iterates over stored data rows (`catpanel(alldata)`)
3. Includes a row if the pattern appears anywhere in the tab-separated line (`string match -nocase`)
4. Repopulates the table with matching rows only
5. Updates the status bar with the count of matching rows

6.1.5 CatalogPanelAutoExtract

A hook procedure called from `fits.tcl` after a FITS file is loaded:

```
proc CatalogPanelAutoExtract {} {
    global catpanel
    if {[info exists catpanel(tbl)]} {
        after 500 CatalogPanelExtract
    }
}
```

The 500 ms delay (`after 500`) ensures the image is fully loaded and rendered before extraction begins.

6.2 Global Variables

Variable	Purpose
`ds9(toppw)`	The top-level `ttk::panedwindow` widget path
`ds9(catalog_frame)`	Frame widget path for the right (catalog) pane
`catpanel(tbl)`	The Tktable widget path
`catpanel(tbldb)`	Name of the global array backing the table data
`catpanel(status)`	Status bar text variable
`catpanel(search_var)`	Filter entry text variable
`catpanel(alldata)`	Raw TSV string from last extraction (for filtering)
`catpanel(filename)`	Last extracted FITS filename
`catpanel(delim)`	Column delimiter (always "\t")

7 Python Extractor Details

7.1 Module Structure

The `ds9_sextract` script (`ds9_sextract.py`) is a single-file Python module with one main function:

```
def extract_sources(fits_file,
                    detect_thresh=1.5,
                    detect_minarea=5,
                    deblend_nthresh=32,
                    deblend_mincont=0.005,
                    phot_aperture=5.0,
                    mag_zeropoint=25.0,
                    gain=0.0,
                    pixel_scale=1.0,
                    seeing_fwhm=3.0,
                    back_size=64,
                    back_filtersize=3):
```

7.2 FITS File Handling

The FITS reader searches for a 2D image HDU in the following order:

1. Iterate through all HDUs; use the first one with 2D data (`hdu.data.ndim == 2`)
2. If no 2D HDU is found, try the primary HDU
3. If the primary has > 2 dimensions, extract the first 2D slice (`data[0]`)
4. The data is converted to `float64` C-contiguous format (required by SEP)

7.3 Automatic Header Parsing

Several parameters are automatically read from the FITS header when not specified by the user:

Parameter	Header Keywords	Fallback
Gain	`GAIN`, `EGAIN`, `CCDGAIN`	0 (Poisson noise disabled)
Saturation level	`SATURATE`, `SATUR`, `SATULEVEL`, `DATAMAX`	65535
Pixel scale	WCS `proj_plane_pixel_scales()`	1.0 arcsec/pixel

7.4 Dependencies

Package	Version	Purpose
`sep`	≥ 1.0	Core source extraction: background estimation, detection, deblending, photometry, and morphology. Python binding of the C library derived from SExtractor.
`astropy`	≥ 4.0	FITS file I/O (`astropy.io.fits`) and WCS coordinate transformations (`astropy.wcs.WCS`).
`numpy`	≥ 1.20	Array operations, mathematical functions, and data type management.

8 Image–Catalog Interaction

This chapter describes the interactive features that link the image display with the catalog panel, enabling bidirectional navigation, source merging, and advanced filtering.

8.1 Overview of Interaction Features

The following features connect the image markers to the catalog table:

Feature	Description
Mark All	Places yellow ellipse markers on all detected sources in the image
Marker Click	Clicking a yellow marker in the image selects and scrolls to the corresponding row in the catalog table, and shows cyan cross + green ellipse markers at the source position
Visible Filter	Toggles display to show only sources within the current viewport
Ctrl+Click Merge	Ctrl+Click on markers to select merge candidates (red thick ellipses); Ctrl+M merges them into a single source
Trim	Column-based min/max filtering with a dialog for each catalog column
Settings	Adjustable extraction parameters (threshold, deblending, photometry, etc.)

8.1.1 Title Bar Layout

```
[Source Extractor]  [Extract] [Settings...] [Mark All] [
    Visible] [Trim...] [Clear]
```

8.2 Mark All Sources

The **Mark All** button creates yellow ellipse markers on all detected sources using the `ISO_RADIUS`, `B_IMAGE/A_IMAGE` ratio, and `THETA_IMAGE` parameters:

Marker region string

```
ellipse($x $y ${semi_a}i ${semi_b}i $theta)
  # color=yellow width=1 tag={sextract_all} tag={
     sextract_src.$num}
    select=0 edit=0 move=0 rotate=0 delete=1 highlite=1
    callback=highlite CatalogPanelMarkerCB {$num}
    callback=unhighlite CatalogPanelMarkerUnCB {$num}
```

Each marker has:

- `sextract_all` tag for bulk operations (deletion, etc.)
- `sextract_src.`*`NUMBER`* tag for individual identification
- `highlite=1` with callbacks for mouse interaction in `catalog` mode

8.3 Marker Click — Image to Catalog Navigation

Clicking a yellow ellipse marker in the image navigates the catalog to the corresponding source.

8.3.1 Implementation

DS9's built-in `highlite` callbacks only fire in `catalog` mode. Since the default viewing mode is `none`, explicit click handlers were added in `frame.tcl`:

frame.tcl — Button1Frame (none mode)

```
none {
    if {$which == $current(frame)} {
        CatalogPanelMarkerClick $which $x $y
    } else {
        ...
    }
}
```

The click detection uses DS9's marker API:

CatalogPanelMarkerClick

```
proc CatalogPanelMarkerClick {which x y} {
    set id [$which get marker catalog id $x $y]
    if {$id == 0} return
    set tags [$which get marker catalog $id tag]
    foreach tag $tags {
        if {[string match "sextract_src.*" $tag]} {
            set src_num [string range $tag 13 end]
            break
        }
    }
    CatalogPanelMarkerCB $src_num $id
}
```

`CatalogPanelMarkerCB` then:

1. Finds the `NUMBER` column in the table header
2. Searches all rows for the matching source number
3. Selects the row and scrolls the table to it
4. Calls `CatalogPanelGotoSource` to pan the image and show selection markers (cyan cross + green ellipse)

8.4 Visible Filter

The **Visible** button toggles between showing all sources and showing only sources within the current viewport.

8.4.1 Viewport Calculation

Viewport bounds computation

```
set center [$frame get cursor image]    ;# {cx cy}
set zoom   [$frame get zoom]            ;# {zx zy}
set cw [winfo width $ds9(canvas)]
set ch [winfo height $ds9(canvas)]
set x_min [expr {$cx - $cw / 2.0 / $zx}]
set x_max [expr {$cx + $cw / 2.0 / $zx}]
set y_min [expr {$cy - $ch / 2.0 / $zy}]
set y_max [expr {$cy + $ch / 2.0 / $zy}]
```

Sources with `X_IMAGE` and `Y_IMAGE` within these bounds are displayed. The status bar shows "Visible: N of M sources in current view."

8.5 Source Merging (Ctrl+Click + Ctrl+M)

Multiple sources can be merged into a single source through a two-step process.

8.5.1 Step 1: Select Merge Candidates

Ctrl+Click on a yellow ellipse marker toggles it as a merge candidate:

- Selected: A red thick ellipse (`width=3`) is overlaid, tagged `sextract_merge` and `sextract_merge.`*`NUMBER`*
- Deselected: Ctrl+Click again removes the red marker and the source from the merge list
- Status bar: "Merge: N sources selected (Ctrl+M to merge, Esc to cancel)"

8.5.2 Step 2: Execute Merge (Ctrl+M)

When at least 2 sources are selected, **Ctrl+M** merges them:

Property	Merge Algorithm
`X_IMAGE`, `Y_IMAGE`	Flux-weighted centroid: $\bar{x} = \sum F_i x_i / \sum F_i$
`FLUX_AUTO`	Sum of all fluxes: $F_{\text{total}} = \sum F_i$
`MAG_AUTO`	Recomputed: $m = -2.5 \log_{10}(F_{\text{total}}) + m_0$

Property	Merge Algorithm
`NPIX_ISO`	Sum of all pixel counts
`ISO_RADIUS`	$r = \sqrt{N_{\text{pix,total}}/(\pi \cdot b/a)}$
`A_IMAGE`, `B_IMAGE`, `THETA_IMAGE`	Flux-weighted average
`NUMBER`	New number: $\max(\text{all NUMBERs}) + 1$
Other columns	Copied from the brightest source

After merging:

1. The original sources are removed from `catpanel(alldata)`
2. The merged source row is appended
3. The table and markers are regenerated
4. The merged source is auto-selected and the image pans to its position
5. Status bar: "Merged N sources into #NEW: pos=(X,Y) mag=M"

8.5.3 Cancel (Escape)

Pressing **Escape** cancels the merge selection, removes all red markers, and clears the merge list.

8.6 Trim — Column Filter Dialog

The **Trim...** button opens a modal dialog for per-column min/max filtering.

8.6.1 Dialog Layout

Trim dialog structure

```
+-----------------------------------------+
|  Trim / Column Filter                   |
+-----------------------------------------+
|  Column        |  Min       |  Max      |
|  NUMBER        | [______]   | [______]  |
|  X_IMAGE       | [______]   | [______]  |
|  Y_IMAGE       | [______]   | [______]  |
|  ...           | ...        | ...       |
|  MAG_AUTO      | [______]   | [______]  |
+-----------------------------------------+
|  [Apply] [Reset] [Save] [Load] [Cancel] |
+-----------------------------------------+
```

- Empty fields mean no constraint
- **Apply**: filter rows where min $\leq$ value $\leq$ max for each specified column, then close dialog and regenerate markers
- **Reset**: clear all min/max fields
- **Save/Load**: persist conditions to `~/.ds9/sextract_trim.prf`
- **Cancel**: close without changes

Status bar after Apply: "Trimmed: N of M sources match conditions."

8.7 Settings Dialog

The **Settings...** button opens a modal dialog to adjust extraction parameters before running Extract.

Parameter	Default	Description
Detect Threshold	1.5	Detection threshold (σ)
Detect Min Area	5	Minimum connected pixels
Deblend NThresh	32	Number of deblending sub-thresholds
Deblend MinCont	0.005	Minimum deblending contrast
Phot Aperture	5.0	Fixed aperture diameter (pixels)
Mag Zeropoint	25.0	Magnitude zero-point m_0
Gain	0.0	Detector gain (e^-/ADU)
Pixel Scale	1.0	Pixel scale (arcsec/pixel)
Seeing FWHM	3.0	Seeing FWHM (arcsec)
Back Size	64	Background mesh size (pixels)
Back Filter Size	3	Background filter kernel (mesh cells)

Settings are saved to / loaded from `~/.ds9/sextract.prf`. The **Defaults** button restores all parameters to their default values.

8.8 Marker Tag Reference

Tag	Description
`sextract_all`	All yellow ellipse markers from Mark All
`sextract_src.`*N*	Individual source marker (N = source NUMBER)
`sextract_sel`	Current selection markers (cyan cross + green ellipse)
`sextract_merge`	All red merge-candidate markers
`sextract_merge.`*N*	Individual merge-candidate marker

8.9 Key Bindings

Key	Action
Click on marker	Navigate catalog to source, show selection markers
Ctrl+Click on marker	Toggle merge candidate (red ellipse)
Ctrl+M	Merge selected sources
Escape	Cancel merge selection
Click table header	Sort by column (toggle ascending/descending)
Click table row	Pan image to source position

9 Building ds9_extract for Each Platform

The ds9_extract binary is a standalone C program that uses the SEP library and CFITSIO for source extraction. It must be compiled separately for each target platform and placed in the bin/ directory alongside the ds9 executable.

9.1 Dependencies

Library	Purpose	Install
CFITSIO	FITS file I/O	Package manager or source
SEP	Source extraction algorithm	Included in ds9_extract.c or separate
libm	Math functions	System standard

9.2 Linux (x86_64)

Install dependencies (Ubuntu/Debian)

```
sudo apt-get install libcfitsio-dev
```

Install dependencies (RHEL/CentOS/Rocky)

```
sudo dnf install cfitsio-devel
```

Install dependencies (conda)

```
conda install -c conda-forge cfitsio
```

Compile

```
gcc -O2 -o ds9_extract ds9_extract.c \
    -I/usr/include -L/usr/lib64 \
    -lcfitsio -lm
# Or with conda:
gcc -O2 -o ds9_extract ds9_extract.c \
    -I$CONDA_PREFIX/include -L$CONDA_PREFIX/lib \
    -lcfitsio -lm -Wl,-rpath,$CONDA_PREFIX/lib
cp ds9_extract /path/to/SAOImageDS9/bin/
```

9.3 macOS (Intel and Apple Silicon)

Install dependencies (Homebrew)

```
brew install cfitsio
```

Install dependencies (conda)

```
conda install -c conda-forge cfitsio
```

Compile (Homebrew)

```
# Intel Mac:
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I/usr/local/include -L/usr/local/lib \
    -lcfitsio -lm

# Apple Silicon (M1/M2/M3):
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I/opt/homebrew/include -L/opt/homebrew/lib \
    -lcfitsio -lm
```

Compile (conda)

```
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I$CONDA_PREFIX/include -L$CONDA_PREFIX/lib \
    -lcfitsio -lm -Wl,-rpath,$CONDA_PREFIX/lib
```

Place binary

```
cp ds9_sextract /path/to/SAOImageDS9/bin/
```

Important

On macOS, if you encounter "library not loaded" errors at runtime, ensure the CFITSIO library path is embedded using `-Wl,-rpath` or set `DYLD_LIBRARY_PATH` before launching DS9.

9.4 Windows

9.4.1 Using MSYS2/MinGW-w64

Install MSYS2 and dependencies

```
# In MSYS2 MinGW64 shell:
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-cfitsio
```

Compile

```
gcc -O2 -o ds9_sextract.exe ds9_sextract.c \
    -I/mingw64/include -L/mingw64/lib \
    -lcfitsio -lm
```

Deploy

```
# Copy binary and required DLLs to ds9 bin directory
cp ds9_sextract.exe /path/to/SAOImageDS9/bin/
cp /mingw64/bin/libcfitsio*.dll /path/to/SAOImageDS9/bin/
cp /mingw64/bin/zlib1.dll /path/to/SAOImageDS9/bin/
```

9.4.2 Using Visual Studio

Compile with MSVC

```
cl /O2 /Fe:ds9_sextract.exe ds9_sextract.c ^
   /I"C:\cfitsio\include" ^
   /link /LIBPATH:"C:\cfitsio\lib" cfitsio.lib
```

Important

On Windows, required DLLs (`libcfitsio*.dll`, `zlib1.dll`) must be placed in the same directory as `ds9_sextract.exe`, or their location must be in the system `PATH`.

9.5 Cross-Platform Tcl Integration

The `CatalogPanelExtract` procedure in `layout.tcl` automatically handles platform differences:

Platform-aware binary discovery

```
set os $::tcl_platform(os)
if {$os eq "Windows NT"} {
    set sextract [file join $bindir ds9_sextract.exe]
} else {
    set sextract [file join $bindir ds9_sextract]
}
```

Platform-aware library path

```
if {$os eq "Darwin"} {
    # macOS: set DYLD_LIBRARY_PATH
    set ::env(DYLD_LIBRARY_PATH) [join $libpaths :]
} elseif {$os ne "Windows NT"} {
    # Linux/Unix: set LD_LIBRARY_PATH
    set ::env(LD_LIBRARY_PATH) [join $libpaths :]
}
# Windows: DLLs found via PATH automatically
```

Cross-platform execution

```
# Direct exec without sh -c (works on all platforms)
exec $sextract $fn {*}$paramargs 2>@stderr
```

	Linux	macOS	Windows
Binary name	ds9_sextract	ds9_sextract	ds9_sextract.exe
Library path env	LD_LIBRARY_PATH	DYLD_LIBRARY_PATH	PATH
Shell required	No	No	No

9.6 Verification

After building for any platform, verify the binary works:

Test extraction

```
./ds9_sextract test_image.fits | head -5
```

Expected output: a tab-separated header line followed by data rows. If library errors occur, check the library path configuration for your platform.